Introduction

Saturday, October 11, 2025 3:36 PM

Introduction to Software Engineering

- Software is a set of computer programs and associated documentation.
- **Software Engineering** is the application of sound engineering principles to produce **economically reliable software** that runs efficiently on real machines.
- It is an **engineering discipline** concerned with all aspects of software production.

Software Types

- **Generic Software:**
  - Developed for general-purpose use by a **large number of users**.
  - Development team controls the process.
  - Example: Word processing software (Microsoft Word).
- **Custom Software:**
  - Developed for a **specific customer's needs**.
  - The customer controls the development process.
  - Example: Control system software for an electronic device.

Real-life Example:

- A **mobile banking app** developed for a single bank is custom software, whereas **Microsoft Office** is generic software used by millions globally.

Attributes of Good Software

- **Maintainability:**
  - Software should evolve to meet **changing customer needs**.
  - Example: Updating an app to support new devices.
- **Dependability:**
  - Reflects **trustworthiness**; system should operate as expected without failure.
  - Example: ATM software reliably dispensing cash correctly.
- **Portability:**
  - Software should be **easy to use across different environments**.
  - Example: A mobile app running on both Android and iOS.
- **Efficiency:**
  - Minimal use of **system resources** like memory and processing time.
  - Example: A video player app that streams without lag or high CPU usage.
- **Usability:**
  - Software should be **easy to use**, with adequate documentation and user-friendly interface.
  - Example: A navigation app providing intuitive maps and clear directions.

Advantages of Software Engineering

- Improved **quality**
- Improved **reliability**
- Increased **productivity**
- Better **requirement specification**
- More accurate **cost and schedule estimates**
- Well-defined **process**

Importance of Software Engineering

- Modern economies depend heavily on **software-controlled systems**.
- **Software expenditure** represents a significant fraction of GNP in developed nations.

Real-life Example:

- Online retail platforms like **Amazon** rely on software engineering to ensure **reliable transactions, efficient inventory management, and good user experience**.

Fundamental Software Engineering Activities

Software engineering involves many types of processes, but **all software processes include four fundamental activities**:

1. Software Specification

- **Definition:** Process of understanding and defining what services the system must provide.
- **Purpose:** Identify system operations and development requirements.
- **Importance:** Critical stage; errors here can cause problems in later design and implementation.

Example:

- Defining features for an online shopping app: user login, product search, payment, and order tracking.

2. Development

- **Definition:** Converting design and implementation into an **executable system**.
- **Includes:** Coding, integrating components, and creating a working software product.

Example:

- Writing code for a mobile banking app based on the specifications, then compiling and testing it to ensure it runs correctly.

3. Validation

- **Definition:** Ensures **the right system is built**.
- **Purpose:** Show that the system **conforms to specifications** and meets **user expectations**.

Example:

- Testing an e-commerce website to ensure checkout works correctly and discounts are applied as expected.

4. Evolution

- **Definition:** Maintenance and updates of the system to meet **changing customer needs** over time.
- **Purpose:** Software should be written to **adapt and evolve**.

Example:

- Updating a social media app to support new features like stories or live video.

Difference Between Software Engineering and Computer Science

Software Engineering	Computer Science
Study of how **software systems are built** and used.	Study of what **computers perform**.
Focuses on the **study and application of software only**.	Focuses on both **software and computational principles**.
Structured process: check, verify, find errors, and remove bugs.	Less structured; requires proper study before executing tasks.
Areas include **software development, testing, and quality assurance**.	Areas include **AI, databases, algorithms, etc.**
Defines **architecture and structural properties** of software.	Studies **computing principles and concepts**.

Example to understand difference:

- **Software Engineering:** Building a reliable online banking system with proper testing, architecture, and maintenance.
- **Computer Science:** Studying algorithms to optimize transaction processing in the banking system.

Difference Between Software Engineering and System Engineering

Software Engineering	System Engineering
Software engineering is an engineering discipline concerned with **all aspects of software production**.	System engineering is a field of engineering and management focused on **designing and managing complex systems over their life cycle**.

Highly focuses on **implementing quality software**.	Highly focuses on **user needs and the broader system**, including software, hardware, and processes.
Includes computer science or computer-based engineering background.	May include **mechanical, electrical, or other engineering disciplines**.
Focuses solely on **software components**.	Focuses on **hardware, software, and integrated system components**.
Newly developed discipline.	Older, established discipline.

Example:

- **Software Engineering:** Developing a mobile banking app with secure transactions and user-friendly design.
- **System Engineering:** Designing an automated airport control system integrating radar, communication, and software components.

Challenges of Software Engineering

1. **Heterogeneity Challenge**
   - **Definition:** Different types of computers and support systems require software to be flexible.
   - **Example:** A video editing software should run on Windows, macOS, and Linux without issues.
2. **Legacy Challenge**
   - **Definition:** Maintaining and updating old software without excessive cost while ensuring essential services continue.
   - **Example:** Updating a bank's core transaction system that has been running for decades.
3. **Delivery Challenge**
   - **Definition:** Businesses demand software quickly, but quality cannot be compromised.
   - **Example:** Rapid release of an e-commerce website for a festival sale.
4. **Trust Challenge**
   - **Definition:** Users must trust software in all aspects of life.
   - **Example:** Medical software that monitors heart conditions must provide accurate and reliable data.
5. **Risk Challenge**
   - **Definition:** Software failure in safety-critical areas can have catastrophic consequences.
   - **Example:** Navigation software failure in airplanes or space systems.
6. **Complexity Challenge**
   - **Definition:** Modern applications are increasingly complex, requiring advanced software engineering techniques.
   - **Example:** Autonomous vehicle software integrating sensors, AI, and cloud services.

Cost of Software Engineering

- **Cost Distribution:**
  - Costs vary depending on **software type** and **development process**.
  - **Example:** Real-time software (like aircraft control systems) requires more testing than web-based systems.
  - Approximate distribution:
    - 60% development costs
    - 40% testing costs
  - For **customer-specific software**, evolution and maintenance costs can exceed initial development costs.
- **Development Model Influence:**
  - The choice of software process model (e.g., Waterfall, Agile) affects cost allocation.

Example: Developing a web-based e-commerce platform may spend more on development than testing, while an embedded medical device software may spend heavily on validation and testing.

Professional Software Development

- **Definition:** Development performed for business or professional purposes, not just personal interest.
- **Components of a Professional Software System:**
  - Multiple programs and configuration files
  - Documentation:
    - Technical documentation (system structure)

        - User documentation (how to use the system)
        - Websites for updates and downloads
- **Goal:** Support long-term maintenance and evolution of software.

Example: A banking software system may include backend services, frontend apps, configuration files for deployment, and user manuals.

Software Engineering Diversity

- **Principle:** No single method suits all systems or companies.
- **Reason:** Different applications require different techniques and tools.
- **Implication:** Engineers must select methods based on the software type.

Example: Agile may suit a startup web application, while V-Model may suit safety-critical software in aerospace.

Reuse

- **Definition:** Maximize efficiency by reusing existing software components where appropriate.
- **Example:** Using a pre-built authentication module across multiple applications instead of coding from scratch.

Internet Software Engineering

Internet software engineering focuses on **developing and managing software applications that run over the internet**, rather than on a single local computer. This approach changes how software is designed, deployed, and maintained.

Key Points

- **Platform**: The internet becomes the platform for running applications.
- **Access**: Applications can be accessed through **web browsers** instead of installing software on each PC.
- **Maintenance**: Software can be **updated centrally**, reducing the need to update every user's machine.
- **Cost Efficiency**: Cheaper to deploy and maintain because users don't need individual installations.
- **Examples**:
    - Web-based email (Gmail, Outlook online)
    - Online office suites (Google Docs, Office 365)
    - Cloud storage services (Dropbox, Google Drive)

Software Engineering Ethics

Software engineering ethics are **principles that guide professional conduct in software development** to ensure responsibility, trust, and legality. These ethics are essential for producing reliable and safe software.

1. **Confidentiality:** Respect client or employee data, whether under a formal agreement or not.
    - *Example:* Not sharing company source code with external parties.
2. **Competence:** Only accept work within your skill level; do not misrepresent abilities.
    - *Example:* A junior developer should not promise to implement a complex AI algorithm alone.
3. **Intellectual Property Rights:** Follow local laws regarding patents, copyrights, and software licensing.
    - *Example:* Avoid using proprietary libraries without proper licensing.
4. **Computer Misuse:** Do not use technical skills to misuse computers.
    - *Example:* Avoid installing malware or using employer systems for personal gaming.